Clase 1, Parte 4: Cierre y Preguntas

Duración Estimada: 30 minutos

Hemos llegado al final de nuestra primera clase. En esta última parte, haremos un resumen de lo aprendido, les daré la tarea para la casa y responderemos a todas las preguntas que tengan.

4.1 Resumen y Tarea para la Casa

Puntos Clave de la Clase:

- **Entorno de Desarrollo:** Hemos configurado un entorno profesional y aislado con VS Code y venv. Esto es el estándar en la industria.
- **Fundamentos de Python:** Reforzamos el uso de diccionarios y listas, y practicamos la lógica con bucles y condicionales.
- **Manejo de Archivos:** Trabajamos con formatos comunes como CSV y JSON, y aprendimos a convertir datos entre ellos de forma segura usando *Context Managers*.
- **Control de Errores:** Implementamos la validación de datos usando bloques try/except, una habilidad crucial para crear aplicaciones robustas.

Tarea para la Casa (Modularización):

Su tarea es refinar el ejercicio del validador de datos para aplicar conceptos de arquitectura y separación de responsabilidades.

1. Tomen el código de la función validar_datos() que creamos en la Parte 3.
2. Modifíquenla para que, si el archivo es válido, devuelva la lista de productos. Si no lo es, que devuelva None.
3. Creen un **nuevo archivo** llamado validar_productos.py. Muevan la función a este archivo.
4. En app.py, **importen** la función desde validar_productos.py y úsenla para validar su archivo. Esto los introduce al concepto de modularización, que será clave para el resto del curso.

Vista Previa de la Próxima Clase:

El próximo día nos adentraremos en la Programación Orientada a Objetos (POO). Este paradigma es la base de frameworks como Django, y es fundamental para entender cómo funcionan los modelos, vistas y ORMs. Asegúrense de tener el entorno virtual de hoy listo, ya que lo usaremos.

4.2 Código Final de los Archivos

Para referencia, aquí tienen el código completo resultante tras realizar la tarea.

app.py

```
import csv
```

```
import json
from validar_productos import validar_datos

def calcular_promedio_precio(lista):
    if not lista:
        return 0
    total_precio = sum(p['precio'] for p in lista)
    return total_precio / len(lista)

# ----- Bloque 1 y 2: Manipulación de Datos en memoria y CSV -----
productos_desde_csv = []
try:
    with open('datos.csv', mode='r', encoding='utf-8') as archivo_csv:
        lector_diccionario = csv.DictReader(archivo_csv)
        for fila in lector_diccionario:
            fila['id'] = int(fila['id'])
            fila['precio'] = int(fila['precio'])
            fila['stock'] = int(fila['stock'])
            productos_desde_csv.append(fila)
except FileNotFoundError:
    print("Error: El archivo 'datos.csv' no se encontró.")

# ----- Bloque 3: Conversión de datos a JSON y Validación -----
if productos_desde_csv:
    datos_json = json.dumps(productos_desde_csv, indent=4)
    with open('salida.json', 'w') as archivo_salida:
        archivo_salida.write(datos_json)

    print("\nArchivo 'salida.json' creado con éxito.")

    print("\n--- Ejecutando el validador modular ---")
    datos_validados = validar_datos('salida.json')
    if datos_validados:
        print("Datos validados correctamente:", datos_validados)
```

validar_productos.py

```
import json

def validar_datos(nombre_archivo):
    """
    Valida un archivo JSON, verificando que el formato sea correcto
```

```
y que los precios sean valores numéricos.
Retorna la lista de datos si es válida, de lo contrario, retorna None.
"""
try:
    with open(nombre_archivo, 'r') as archivo_json:
        datos = json.load(archivo_json)

    if not isinstance(datos, list):
        raise ValueError("El archivo no contiene una lista de datos.")

    for item in datos:
        if not isinstance(item.get('precio'), (int, float)):
            raise TypeError(f"El precio del producto '{item.get('nombre')}' no es numérico.")

    print("Validación exitosa.")
    return datos

except FileNotFoundError:
    print(f"Error: El archivo '{nombre_archivo}' no se encontró.")
    return None
except json.JSONDecodeError:
    print("Error: El archivo no tiene un formato JSON válido.")
    return None
except (ValueError, TypeError) as e:
    print(f"Error en la validación de datos: {e}")
    return None
```